Conceptos de POO

Este documento es simplemente una serie de conceptos que se va a repetir a lo largo de estos documentos. Evidentemente, están muy relacionados con lo que es la Programación, la Orientación a Objetos, etc. Algunos son bastante raros y difíciles de poner en contexto, pero vamos, de eso va esta serie de documentos… ¿no?

Herencia

Ya deberías de saber lo que es la **Herencia**. Es un mecanismo que permite crear nuevas clases basadas en otras ya existentes. Su utilidad explicada de forma sencilla es evita repetir código y facilita la reutilización. Las clases hijas heredan atributos y métodos de la clase padre, pudiendo además añadir o modificar comportamientos.

Explicado brevemente, supongamos que tenemos Coche:

```
public class Coche (){

    private String motor = null;

    public void conducir (){
        ...
    }
}
```

Coche tiene un atributo (motor) y un método (conducir). También tiene un comportamiento propio, es decir, que un coche se conduce de cierta manera (la implementación del método conducir ()).

Si tuviésemos que añadir un Coche de Carreras a nuestra aplicación, lo correcto NO sería añadir una nueva clase, sino una extensión de la clase Coche. Por ejemplo:

```
public class CocheDeCarreras extends Coche (){

    public void conducir (){
        ...
    }
}
```

Coche de Carreras tendría su propio comportamiento, a la vez que hereda características que puede aprovechar de Coche. En este caso, decimos que Coche de Carreras es “un tipo de” Coche.

Una de las máximas del diseño de aplicaciones dice:

"Tus programas deben ser abiertos a las extensiones, pero cerrados a las modificaciones"

Lo que significa que, tus aplicaciones deben de prepararse para poder aceptar Herencia de forma natural, pero (y esto lo entenderás más adelante) que no necesiten hacer reescrituras de código cada vez que añadas clases nuevas.

Polimorfismo

El polimorfismo permite usar **un mismo método** para objetos de distintos tipos. Esto se consigue de diferentes formas. La más sencilla es mediante Herencia. En el ejemplo de los coches, definimos un método conducir a la clase Coche. Luego definimos Coche de Carreras y Coche Todo Terreno como extensiones de Coche. Esto hace que:

```
CocheDeCarreras cocheDeCarreras = new CocheDeCarreras ();
CocheTodoTerreno cocheTodoTerreno = new CocheTodoTerreno ();

cocheDeCarreras.conducir ();
cocheTodoTerreno.conducir ();
```

Luego, mismo método conducir (), diferentes comportamientos. Esta técnica permite escribir código más flexible y extensible, ya que así podemos llamar a ese método sin importar de qué tipo sea el objeto.

Por supuesto, hay varias formas de Polimorfismo:

- **Sobrecarga**: Existen varios métodos con el mismo nombre, pero distinta implementación dentro de la misma clase o en subclases. El compilador elige cuál usar según los parámetros en el momento de la compilación.

  Es el ejemplo de más arriba.

- **Paramétrico**: Existen funciones con el mismo nombre, pero diferentes parámetros.

  Un ejemplo sencillo:

```
public void conducir (){
    ...
}

public void conducir (int velocidad){
    ...
}
```

- **Inclusión**: Es cuando se puede llamar a un método sin tener que conocer su tipo. Por ejemplo, en este caso:

```
Coche coche = new CocheDeCarreras ();
coche.conducir ();
```

  Obviamente, esto funciona porque Coche de Carreras extiende de Coche (), y nos permite llamar a conducir ().

Encapsulación

La **encapsulación** consiste en ocultar los detalles internos de una clase. De esta forma, alguien que quiera usar la clase solamente tiene que conocer cómo usarla, es decir, sus métodos públicos. Es posible ocultar tanto métodos como propiedades de una clase, en este caso, dejando solamente los métodos públicos getters y setters que nos interesan.

Por ejemplo, la clase Coche de Carreras tiene cincuenta métodos y veinte atributos. Para evitar errores, inconsistencias, etc. optamos por encapsular el código, dejando únicamente como públicos a los métodos que necesita quien use esta clase: conducir (), acelerar (), getMotor (), etc.

La encapsulación NO se hace solo a nivel de clase. Se puede hacer a nivel de paquete, o a nivel de método. La primera implica hacer clases privadas para encapsular a nivel de paquete o módulo; mientras que la segunda es prescindir de variables de clase para proceder solamente con variables locales dentro de métodos.

Abstracción

La **abstracción** consiste en centrarse en qué hace un objeto. Cuando defines una clase abstracta estás declarando qué hace la clase, pero ignoras cómo lo hace. Porque no importa. Obviamente, esto se usa mucho con Herencia para definir comportamientos.

Por ejemplo, podríamos definir Coche como abstracta:

```
public abstract class Coche (){

    private String motor = null;

    public void conducir ();
}
```

Así, estaríamos especificando qué hace un coche, pero sin decir cómo lo hace internamente. Obviamente, NO puedes instanciar una clase abstracta por este motivo. A veces, de forma chapucera, se usa la abstracción para evitar que un programador despistado instancie una clase; pero no es su uso principal.

Interfaces

Una **interfaz** define un conjunto de métodos que una clase debe implementar. Es un contrato con esas clases. Al 'firmarlo', la clase se compromete a implementar esos métodos; pero sin impedirle añadir cosas nuevas. En general, la interfaz se usa para asegurarse de que distintas clases comparten un mismo comportamiento sin importar cómo lo implementan. Es clave para aplicar el polimorfismo y diseñar código desacoplado.

En Java, hay un par de normas a tener en cuenta con las Interfaces:

- Una Interfaz es una clase, y la puedes poner métodos normales. Estos métodos serán accesibles desde las clases que la implementan; pero es un ERROR. No debe hacerse. Nadie espera que un Interfaz tenga código ejecutable, ni lo va a buscar ahí.

- Hay quienes dicen que, para poder hacer Herencia Múltiple en Java, es decir, clases que heredan de varios padres; se pueden usar Interfaces. pero esto es **un parche**. Java NO PERMITE Herencia Múltiple. Y si te ves en la necesidad de usar este truco, tu diseño está mal. Punto.

- A veces, cuando se habla de programación o diseño, se usa la palabra Interfaz para referirse indistintamente a una Clase Abstracta o a una Interfaz. Esto es así porque en muchas ocasiones da lo mismo, y el ejemplo que te están explicando funciona para ambas cosas. Puede que veas esto a lo largo de estos apuntes.

  Por ejemplo, puede ser que se diga **'Interfaz'** para un ejemplo similar a éste, pero en el fondo, se puede estar refiriendo indistintamente a cualquiera de las dos implementaciones. Lo importante del ejemplo, no obstante, es ver cómo se puede retornar un objeto de forma genérica.

```
// Esto es tan válido...
CocheInterface coche = getCocheAsInterface ();

public CocheInterface getCocheAsInterface () {
    return new Coche();
}

// Como esto otro...
CocheAbstract coche = getCocheDeCarreras ();

public CocheAbstract getCocheDeCarreras () {
    return new CocheDeCarreras();
}
```

Código desacoplado

El **código desacoplado** es aquel en el que los componentes (clases, módulos, métodos…) dependen lo mínimo posible unos de otros. De esta forma, cada parte puede modificarse sin afectar demasiado al resto del sistema. Un programa que se diseñe con esta idea en mente será más fácil de mantener, más fácil de extender, y permitirá reutilizar sus partes de forma más eficiente.

Un ejemplo de código acoplado sería:

```
public class Vehiculo {
    private CocheDeCarreras coche = new CocheDeCarreras();

    void conducir() {
        coche.conducir();
    }
}
```

La clase Vehículo depende de Coche de Carreras. En cambio, podemos desacoplarla así:

```
interface CocheInterface {
    void enviar(String mensaje);
}

public class CocheDeCarreras implements CocheInterface {
    public void conducir() {
        ...
    }
}

class Vehiculo {

    private CocheInterface cocheInterface = null;

    Vehiculo(CocheInterface cocheInterface) {
        this.cocheInterface = cocheInterface;
    }

    void conducir() {
        cocheInterface.conducir();
    }
}
```

Ahora, Vehículo no sabe si el coche que tiene que conducir es un Coche de Carreras, o un Coche Todo Terreno, etc. dado que sólo depende de su Interfaz.

No siempre hay que buscar el desacoplamiento total. En general, hay que desacoplar lo que realmente puede cambiar o necesita ser reutilizado.

El patrón **MVC** por ejemplo necesita una clara separación entre capas, lo que implica que desacoplemos las unas de las otras.